

HDOC-30

Aim:

To write a menu-driven C program to perform the following operations on an array:

1. Sort the array using Bubble Sort and search for an element using Binary Search.
2. Perform Insertion and Deletion at a valid position.
3. Rotate the array to the right by k positions.
4. Exit the program.

Task1: Algorithm:

1. Start.
2. Read the number of elements n.
3. Check whether n is valid.
4. Read n array elements.
5. Display the menu.
6. Read the user's choice.
7. Use a switch statement:
 - **Case 1:** Sort the array using Bubble Sort, then read the element to search and perform Binary Search.
 - **Case 2:** Display a sub-menu:
 - Insert an element at a valid position.
 - Delete an element from a valid position.
 - **Case 3:** Read k and rotate the array to the right by k positions.
 - **Case 4:** Exit.
 - Otherwise, display "Invalid choice".
8. Repeat the menu until the user selects Exit.
9. Stop.

Task2: Test case table:

Test Case	Operation	Input Array	Additional Input	Expected Output
1	Bubble Sort + Search	50 20 40 10 30	Search 40	10 20 30 40 50, found at position 4
2	Binary Search	5 10 15 20 25	Search 12	Element not found
3	Insertion	10 20 30 40	Value 25, position 3	10 20 25 30 40
4	Deletion	10 20 30 40 50	Position 3	10 20 40 50
5	Right Rotation	10 20 30 40 50	k = 2	40 50 10 20 30

Task3: C Program:

```
#include <stdio.h>
#define MAX 100
void display(int a[], int n);
void bubbleSort(int a[], int n);
void binarySearch(int a[], int n);
void insertElement(int a[], int *n);
void deleteElement(int a[], int *n);
void rightRotate(int a[], int n);
int main()
{
    int a[MAX];
    int n, i, choice, subChoice;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    while (n <= 0 || n > MAX)
    {
        printf("Invalid size! Enter size between 1 and %d: ", MAX);
        scanf("%d", &n);
    }
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    do
    {
```

```

printf("\n----- ARRAY MENU ----- \n");
printf("1. Bubble Sort and Binary Search\n");
printf("2. Insertion and Deletion\n");
printf("3. Right Rotation by k positions\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice)
{
    case 1:
        bubbleSort(a, n);
        printf("\nArray after Bubble Sort:\n");
        display(a, n);
        binarySearch(a, n);
        break;
    case 2:
        printf("\n--- INSERTION / DELETION ---\n");
        printf("1. Insertion\n");
        printf("2. Deletion\n");
        printf("Enter your choice: ");
        scanf("%d", &subChoice);
        switch (subChoice)
        {
            case 1:
                insertElement(a, &n);
                break;
            case 2:
                deleteElement(a, &n);
                break;
            default:
                printf("Invalid choice!\n");
        }
        break;
    case 3:
        rightRotate(a, n);
        break;
    case 4:
        printf("Exiting program...\n");
        break;
    default:
        printf("Invalid choice!\n");
}
} while (choice != 4);
return 0;
}

```

```

/* Function to display array */
void display(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
}

/* Bubble Sort */
void bubbleSort(int a[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

/* Binary Search */
void binarySearch(int a[], int n)
{
    int key;
    int low = 0;
    int high = n - 1;
    int mid;
    int found = 0;
    printf("Enter element to search: ");
    scanf("%d", &key);
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (a[mid] == key)
        {
            printf("%d found at position %d.\n", key, mid + 1);
            found = 1;
            break;
        }
    }
}

```

```

    }
    else if (key < a[mid])
    {
        high = mid - 1;
    }
    else
    {
        low = mid + 1;
    }
}
if (found == 0)
{
    printf("%d not found in the array.\n", key);
}
}

/* Insertion */
void insertElement(int a[], int *n)
{
    int value, pos, i;
    if (*n >= MAX)
    {
        printf("Array is full. Insertion is not possible.\n");
        return;
    }
    printf("Enter element to insert: ");
    scanf("%d", &value);
    printf("Enter position (1 to %d): ", *n + 1);
    scanf("%d", &pos);
    if (pos < 1 || pos > *n + 1)
    {
        printf("Invalid position!\n");
        return;
    }
    for (i = *n; i >= pos; i--)
    {
        a[i] = a[i - 1];
    }
    a[pos - 1] = value;
    (*n)++;
    printf("Array after insertion:\n");
    display(a, *n);
}

/* Deletion */
void deleteElement(int a[], int *n)
{
    int pos, i;

```

```

if (*n == 0)
{
    printf("Array is empty. Deletion is not possible.\n");
    return;
}
printf("Enter position to delete (1 to %d): ", *n);
scanf("%d", &pos);
if (pos < 1 || pos > *n)
{
    printf("Invalid position!\n");
    return;
}
for (i = pos - 1; i < *n - 1; i++)
{
    a[i] = a[i + 1];
}
(*n)--;
printf("Array after deletion:\n");
display(a, *n);
}
/* Right Rotation */
void rightRotate(int a[], int n)
{
    int k, i, j, temp;
    if (n == 0)
    {
        printf("Array is empty!\n");
        return;
    }
    printf("Enter number of positions to rotate: ");
    scanf("%d", &k);
    if (k < 0)
    {
        printf("Invalid value of k!\n");
        return;
    }
    k = k % n;
    for (i = 1; i <= k; i++)
    {
        temp = a[n - 1];
        for (j = n - 1; j > 0; j--)
        {
            a[j] = a[j - 1];
        }
        a[0] = temp;
    }
}

```

```

    }
    printf("Array after right rotation:\n");
    display(a, n);
}

```

Task4: Output:

```

(kali@kali)~/hdoc30.c]
$ ./a.out
Enter number of elements: 5
Enter 5 elements:
50
20
40
10
30
Press any key to continue...

---ARRAY MENU---
1. Bubble Sort and Binary Searchn2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 1

Array after Bubble Sort:
1020304050
Enter element to search: 40
40 found at position 4.

```

```

(kali@kali)~/hdoc30.c]
$ ./a.out
Enter number of elements: 5
Enter 5 elements:
5
10
15
20
25
Press any key to continue...

---ARRAY MENU---
1. Bubble Sort and Binary Searchn2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 1

Array after Bubble Sort:
510152025
Enter element to search: 12
12 not found in the array.

---ARRAY MENU---
1. Bubble Sort and Binary Searchn2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 4
Exiting program ...

```

```
(kali@kali)-[~/hdoc30.c]
$ ./a.out
Enter number of elements: 5
Enter 5 elements:
10
20
30
40
50

---ARRAY MENU---
1. Bubble Sort and Binary Searchn2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 2

---INSERTION/DELETION---
1. Insertion
2. Deletion
Enter your choice: 3
Invalid choice!

---ARRAY MENU---
1. Bubble Sort and Binary Searchn2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 4
Exiting program...

(kali@kali)-[~/hdoc30.c]
$ ./a.out
Enter number of elements: 5
Enter 5 elements:
10
20
30
40
50

---ARRAY MENU---
1. Bubble Sort and Binary Searchn2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 3
Enter number of position to rotate: 2
Array after right rotation:
4050102030

---ARRAY MENU---
1. Bubble Sort and Binary Searchn2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 4
Exiting program...
```